

PATIENT HEALTH MONITORING SYSTEM

HCLTech Career Shaper Programme – Embedded Systems & IoT

Submitted by	Himanshu Dubey
Institution	ABES Engineering College
Project type	Individual Project
Academic Year	2025–26
Project Duration	4 Weeks

Acknowledgement

I am grateful to HCLTech for the Career Shaper Programme in Embedded Systems & IoT and for the opportunity to complete this individual project. I thank ABES Engineering College for the academic support and learning environment that helped me work on dataset analysis, simulated monitoring, JSON generation, and communication-flow demonstration. I also acknowledge the technical documentation and learning resources used during the project.

Abstract

This report presents my software-based Patient Health Monitoring System project. The proposed design uses an ESP32, MAX30100, DHT11, and OLED, but the current work does not use live hardware. I analysed a dataset of 5,000 records, checked its quality, applied threshold-based alert logic, generated 20 simulated readings, produced JSON messages, and demonstrated simulated publisher/subscriber communication through the topic patient/monitor/data.

The dataset has seven columns: timestamp, patient_id, heart_rate_bpm, body_temp_C, SpO2_pct, comm_channel, and alert_flag. Validation found 0 missing values and the alert logic passed its checks. The approximate averages were 77 BPM, 36.7°C, and 97%; the combined alert distribution was approximately 4%, while the heart-rate-only comparison was approximately 44.6%. The simulation produced 8 normal and 12 alert readings. Notebook graphs of the dataset are included in Chapter 5. This is an academic prototype, not a certified medical device, and it must not be used for diagnosis or treatment.

Keywords: patient monitoring, embedded systems, IoT, dataset analysis, simulated readings, JSON, alert logic.

Chapter 1 — Introduction

Patient monitoring combines sensing, processing, communication, and visualisation. In an embedded and IoT context, measurements are collected, checked against defined limits, and shared in a form that a user or application can understand. In this project, I studied that workflow through software rather than live medical hardware.

The report therefore focuses on the supplied dataset, preprocessing, rule-based alerts, simulation, JSON output, and a simulated publisher/subscriber exchange. Proposed hardware and future implementation are clearly separated from the work completed during the four-week project.

Chapter 2 — Problem Statement, Objectives and Scope

2.1 Problem Statement

A useful monitoring workflow must handle records consistently and identify measurements that satisfy alert conditions. My problem was to prepare patient-monitoring data, validate its important fields, apply the specified limits, and represent each result as structured communication data.

2.2 Objectives

1. Understand the structure and quality of the monitoring dataset.
2. Check timestamps, missing values, duplicates, ordering, communication categories, and measurement ranges.
3. Apply the defined heart-rate, body-temperature, and SpO2 alert conditions.
4. Generate and evaluate 20 simulated readings.
5. Create JSON data and demonstrate the topic patient/monitor/data in a simulated publisher/subscriber flow.
6. Compare the combined alert logic with a heart-rate-only result.

2.3 Scope and Boundaries

The scope covers software analysis, preprocessing, validation, simulation, JSON generation, and communication-flow demonstration. ESP32, MAX30100, DHT11, and OLED are proposed components only. Live sensor readings, live MQTT connectivity, hardware performance, and clinical validation were not completed.

Chapter 3 — Requirements, Technologies and Proposed Architecture

The proposed system is intended to collect heart rate, SpO2, and body temperature, process the values, show a local result, and communicate a structured monitoring event. The implementation completed for this report is software-based; the physical system remains proposed.

Area	Technology	Purpose or status
Controller	ESP32	Proposed embedded controller for processing and communication.
Sensors	MAX30100; DHT11	Proposed heart-rate/SpO2 and body-temperature sensors.
Display	OLED	Proposed local display; not implemented.
Analysis	Python, Jupyter, pandas, NumPy	Dataset inspection, preprocessing, validation, and calculations.
Visualisation	Matplotlib, seaborn	Notebook analysis graphs (Figures 5.1–5.5).
Data exchange	JSON; patient/monitor/data	Structured messages and simulated publisher/subscriber flow.

3.1 Proposed Architecture

The intended flow is: proposed sensors → ESP32 → processing and threshold checks → OLED display and communication message. In this project, dataset values and simulated readings replace live sensor input, while the communication step remains a software demonstration.

3.2 Requirements

Input: monitoring records with patient and communication fields.

Processing: data checks, threshold evaluation, summary statistics, and simulation.

Output: alert status, JSON records, analysis results, and simulated communication messages.

Chapter 4 — Dataset, Preprocessing and Methodology

The dataset contains 5,000 monitoring records and seven columns. I used it as the main source for software analysis, validation, summary values, and comparison of alert rules. The review found 0 missing values.

4.1 Dataset Columns

Column	Meaning
timestamp	Date and time of the monitoring record.
patient_id	Identifier associated with the patient record.
heart_rate_bpm	Heart rate in beats per minute.
body_temp_C	Body temperature in degrees Celsius.
SpO2_pct	Peripheral oxygen saturation as a percentage.
comm_channel	Categorical communication-channel value.
alert_flag	Alert status associated with the record.

4.2 Threshold Conditions

Parameter	Alert when
Heart rate	Below 50 or above 110 BPM.
Body temperature	Below 35.5°C or above 38°C.
SpO2	Below 94%.

4.3 Preprocessing and Method

I inspected the column names and types, checked timestamp handling, reviewed missing values and duplicates, sorted the observations as required, and treated comm_channel as categorical data. I then checked the physiological ranges, applied the threshold rules, calculated approximate averages and alert distributions, generated 20 simulated readings, and prepared JSON outputs.

The approximate dataset averages were 77 BPM for heart rate, 36.7°C for body temperature, and 97% for SpO2. These are descriptive results from the project data, not clinical reference values.

Chapter 5 — Implementation, Alert Logic, Simulation and Results

I implemented the project as a software workflow consisting of dataset loading, quality checks, threshold evaluation, summary analysis, simulated readings, JSON generation, and simulated publisher/subscriber communication. The proposed hardware was not used to produce these results.

5.1 Alert Logic and Communication

A record is marked as an alert when heart rate is below 50 or above 110 BPM, body temperature is below 35.5°C or above 38°C, or SpO2 is below 94%. If none of these conditions applies, the record is treated as normal for this rule set.

Each simulated monitoring event can be represented as JSON and associated with the topic patient/monitor/data. This publisher/subscriber exchange is simulated; live MQTT connectivity was not completed.

5.2 Results and Testing

Check or result	Value	Meaning
Dataset	5,000 records; 7 columns	Source used for analysis.
Missing-value check	0 missing values	Completeness check passed.
Alert logic	Passed	Specified threshold checks worked as intended.
Simulation	20 readings	8 normal and 12 alert readings.
Combined alert distribution	Approximately 4%	Dataset-level result.
Heart-rate-only comparison	Approximately 44.6%	Separate single-parameter comparison.
Approximate averages	77 BPM; 36.7°C; 97%	Heart rate; body temperature; SpO2.

5.3 Notebook Figures

The following graphs were produced in the project notebook from the supplied dataset. They are descriptive and are shown to support the threshold-based analysis; they do not represent live sensor output.

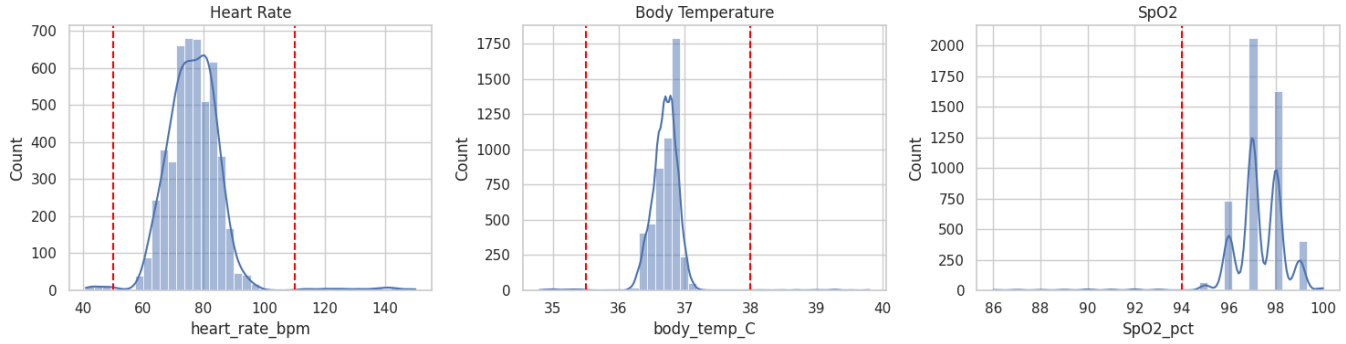


Figure 5.1. *Distributions of heart rate, body temperature, and SpO2. Red dashed lines mark the alert thresholds (50/110 BPM, 35.5/38°C, 94%).*

Most records fall inside the threshold limits: heart rate is concentrated roughly between 60 and 100 BPM, body temperature close to 36.5–37°C, and SpO2 mainly between 96% and 98%. Only a small number of records lie beyond the red threshold lines, consistent with the low combined alert distribution of approximately 4%.

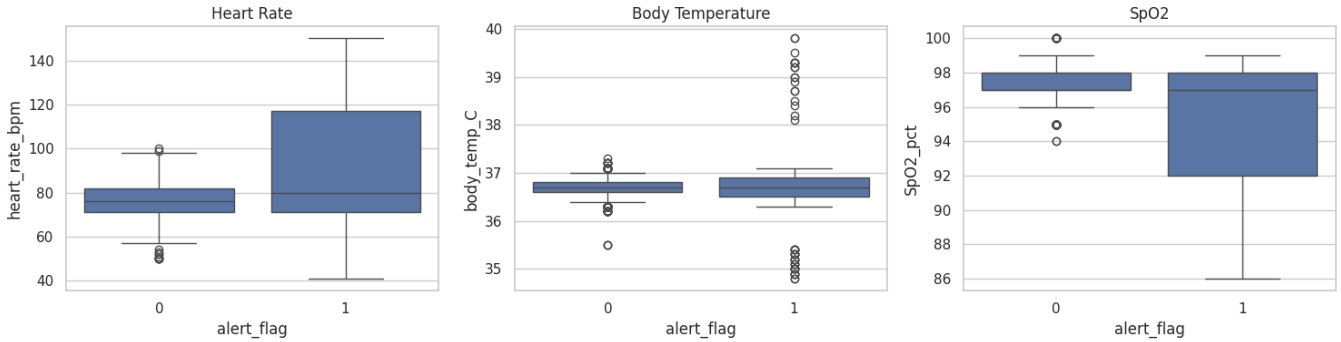


Figure 5.2. *Box plots of heart rate, body temperature, and SpO2 for normal (alert_flag = 0) and alert (alert_flag = 1) records.*

Records with alert_flag = 1 show a much wider spread than normal records. Heart rate in the alert group reaches about 150 BPM and falls to about 40 BPM, SpO2 extends down to about 86%, and body-temperature outliers appear on both the low (about 35°C) and high (about 39–40°C) sides.



Figure 5.3. *Heart rate versus SpO2, coloured by alert_flag. Dotted lines show the heart-rate (50, 110 BPM) and SpO2 (94%) thresholds.*

Alert records (red) lie outside the normal band: below 50 BPM, above 110 BPM, or below 94% SpO2. Normal records (green) cluster inside the limits, which visually supports the alert logic.

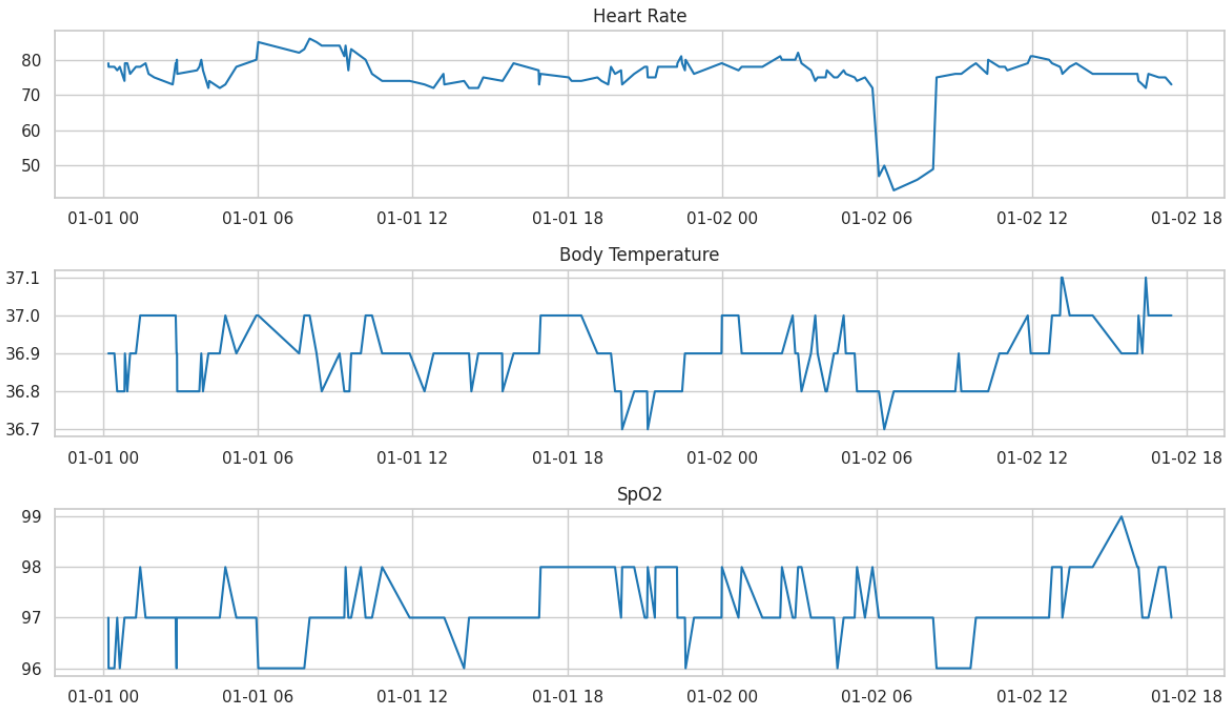


Figure 5.4. *Heart rate, body temperature, and SpO2 plotted against time for the records shown in the notebook.*

The time-series view shows how each parameter changes across the plotted period. Body temperature and SpO2 remain within a narrow range, while heart rate shows a clear drop to roughly 43–50 BPM on 01-02 in the early morning before returning to about 75 BPM.

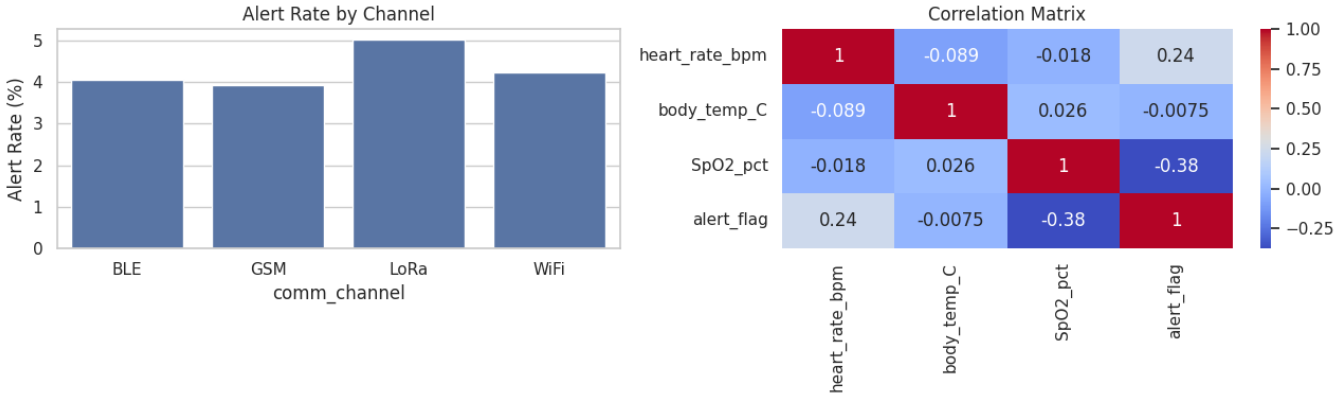


Figure 5.5. *Alert rate by communication channel (left) and correlation matrix of the numeric fields with alert_flag (right).*

Alert rates by channel are all in the region of 4–5%, with LoRa appearing slightly higher than BLE, GSM, and WiFi. In the correlation matrix, alert_flag has a moderate negative correlation with SpO2 (–0.38), a weaker positive correlation with heart rate (0.24), and almost none with body temperature (–0.0075). These are descriptive associations within this dataset only.

The results show that the software workflow can organise records, apply rule-based alerts, and prepare structured messages. They do not demonstrate sensor performance, hardware reliability, network throughput, or clinical accuracy.

Chapter 6 — Testing, Limitations, Future Scope and Conclusion

6.1 Testing and Limitations

The main checks covered timestamps, missing values, duplicates, ordering, categorical communication data, measurement ranges, and the alert boundaries. The completed software results are valid only within the supplied dataset, simulated readings, and stated rules. They do not establish clinical accuracy or device reliability.

ESP32, MAX30100, DHT11, OLED, live sensor readings, live MQTT connectivity, and clinical validation were not completed. Patient-related data should be handled responsibly, and the alert flag should not be interpreted as a diagnosis or treatment recommendation.

6.2 Future Scope

1. Integrate and calibrate the proposed ESP32, MAX30100, DHT11, and OLED hardware.
2. Replace simulated readings with carefully tested sensor readings.
3. Implement secure real communication after defining the required software and security controls.
4. Test boundary values, repeated measurements, sensor failures, and communication interruptions.

6.3 Conclusion

In this four-week project, I developed a software-based monitoring workflow using 5,000 records, seven identified columns, threshold-based alerts, JSON output, and simulated publisher/subscriber communication. The validation found 0 missing values and passed the alert logic. The project produced 20 simulated readings, including 8 normal and 12 alert readings, with an approximately 4% combined alert distribution and an approximately 44.6% heart-rate-only comparison. The work demonstrates the analysis and communication concept while remaining clear that the proposed hardware and clinical system have not yet been implemented.

References

1. Python Software Foundation. Python Documentation. https://docs.python.org/3/
2. pandas development team. pandas Documentation. https://pandas.pydata.org/docs/
3. NumPy developers. NumPy Documentation. https://numpy.org/doc/
4. Matplotlib development team. Matplotlib Documentation. https://matplotlib.org/stable/
5. Waskom, M. seaborn Documentation. https://seaborn.pydata.org/
6. Project Jupyter. Jupyter Documentation. https://docs.jupyter.org/en/latest/
7. Python Software Foundation. json — JSON encoder and decoder. https://docs.python.org/3/library/json.html
8. Espressif Systems. ESP32 Technical Documentation. https://docs.espressif.com/projects/esp-idf/en/latest/esp32/